

Milo, a Fully Autonomous Indoor/Outdoor Robotic Guide Dog

Anonymous Author(s)

Affiliation

Address

email



Figure 1: **Milo** autonomously guides a handler along indoor and outdoor paths while avoiding obstacles and oncoming pedestrians, adapting to the handler’s position, desired walking speed, and turn commands. The project is released as open source.

Abstract:

Many Blind and Low-Vision (BLV) people rely on guide dogs for moment-to-moment navigation, such as staying on path and avoiding obstacles and pedestrians. However, guide dogs are expensive to acquire and maintain (approximately \$50k USD plus ongoing costs), often involve long waiting lists, and have relatively short life expectancies. While robot guide dogs offer a promising alternative, existing approaches exploring this idea suffer from several drawbacks: They often lack the autonomy required for real-world deployment, relying on prior 3D scans of the environment, external computation, or limited awareness of the handler. In this work, we present Milo, the first open-source, low-cost (approximately \$2k USD) robotic guide dog platform capable of fulfilling the basic collaborative navigation role expected of a guide dog. Milo is fully autonomous, requiring no *a priori* knowledge of the environment, completely self-contained with all computation performed onboard, and suitable for both indoor and outdoor navigation while avoiding obstacles and pedestrians. Our system consists of a modified Uni-tree Go2 robot (equipped with onboard compute, sensors, and a handle), a perception stack combining voxel mapping with floor, obstacle, and pedestrian detection, and a navigation stack based on an obstacle-avoidance policy trained in a custom bird’s-eye-view simulator. We evaluate Milo in real indoor and outdoor obstacle courses and compare it against a costmap-based baseline, demonstrating smoother navigation and fewer handler collisions. To maximize accessibility for BLV users, we release both the robot hardware instructions and the complete software stack as open source.

Keywords: Quadruped Robot, Assistive Technology, Obstacle Avoidance

25 1 Introduction

26 According to recent global estimates, approximately 43 million people worldwide are blind and
27 more than 295 million live with moderate-to-severe vision impairment [1]. Many Blind and Low
28 Vision (BLV) individuals rely on white canes, electronic navigation aids, or trained guide dogs for
29 safe and independent mobility. However, just breeding, raising, and training a guide dog can cost
30 more than \$50,000 per animal [2]. Additionally, prospective handlers often face long waiting lists
31 before being matched with a dog. These challenges motivate the development of robotic guide dog
32 systems capable of providing accessible and scalable mobility assistance.

33 A robotic guide dog must safely navigate complex environments by identifying walkable paths,
34 avoiding obstacles and pedestrians, and maintaining comfortable motion for the handler. Several
35 recent works have explored robotic guide dog navigation [3, 4, 5, 6, 7]. However, many existing
36 systems rely on prior maps [4], teleoperation [7], or external computation [5], limiting their auton-
37 omy in real-world deployment. Another limitation is that existing approaches often treat the robot
38 as an independent agent, even though guide dog navigation is inherently collaborative. In practice,
39 the handler provides high-level navigation intent while the guide dog performs local path following
40 and obstacle avoidance [8, 5]. A robotic guide dog should therefore adapt its behavior to the handler
41 rather than operate in isolation.

42 To address these challenges, we propose a robotic guide dog navigation system for previously un-
43 seen indoor and outdoor environments that is capable of operating autonomously on-device. Our
44 approach combines walkable-path segmentation, obstacle avoidance, and reinforcement-learning-
45 based navigation with explicit handler awareness, enabling operation without prior mapping or exter-
46 nal infrastructure. We developed a GPU-accelerated bird’s-eye view (BEV) simulator, implemented
47 in Taichi [9], that simulates a robot guiding a handler along indoor/outdoor sidewalks while avoiding
48 obstacles and pedestrians. To make the system robust and responsive to various handler needs, the
49 navigation policy is trained via Reinforcement Learning (RL) using diverse handler positions. At
50 run time, onboard RGB images, LiDAR scans, odometry, and handle magnetic encoder measure-
51 ments are acquired and processed into a human-interpretable BEV representation that matches the
52 simulator observations. The policy trained in simulation is then deployed zero-shot on a modified
53 quadruped robot. The robot is equipped with a handle mounted through a two-degree-of-freedom
54 passive joint that measures the handler’s relative position. A D-pad on the handle allows the han-
55 dler to adjust the walking speed and issue turn commands. We evaluate our system in previously
56 unseen indoor and outdoor environments on path-following, static obstacle avoidance, pedestrian
57 avoidance. Compared with a costmap-based baseline, which follows paths but does not account for
58 changes in the handler’s position and suffers from blind spots, our BEV-based RL policy retains a
59 geometric memory of the environment and protects the handler from collisions.

60 The main contributions of this work are:

- 61 1. A fully onboard robotic guide dog navigation system capable of operating in previously
62 unseen indoor and outdoor environments without prior mapping, consisting of custom robot
63 hardware for dynamic handler motion and a perception/BEV-mapping system.
- 64 2. An indoor/outdoor BEV simulation environment for efficiently training and evaluating re-
65 inforcement learning policies for path-following and smooth obstacle avoidance policies.
- 66 3. An open-source release of robot hardware design and the codebase including the simulation
67 environment, policy training, onboard perception stack, and pretrained policies.

68 2 Related Work

69 **Hardware Autonomy.** Hardware autonomy is a fundamental requirement for assistive guide robots
70 operating in real-world environments. However, several existing systems still offload perception or
71 planning pipelines to external laptops or remote computation [5]. Recent work has improved on-
72 board integration by embedding compact high-performance computing hardware directly onto the
73 robot platform [6][3], enabling fully onboard navigation without external resources. Similarly, our
74 framework performs the entire pipeline onboard the robot.

Table 1: Comparison with previous robotic guide navigation systems.

Method	Indoor & Outdoor Navigation	Unknown Environment	Fully Onboard	Takes Handler Commands	Dynamic Handler Modeling
Sorokin et al. [3]	Outdoor only	✓	✓	✗	✗
Cai et al. [4], “RDog”	✓	✗	✓	✓	Partial
Hwang et al. [5], “Summer”	Indoor only	✓	✗	✓	Static
Hwang et al. [6], “GuideNav”	Outdoor only	✗	✓	✗	✗
Ours, “Milo”	✓	✓	✓	✓	✓

Perception and Navigation. Previous work has demonstrated autonomous navigation in either indoor [5][10], or outdoor environments [3][11]. Indoor systems often rely primarily on LiDAR data and assume all traversable space is walkable [10], an assumption that does not generalize well outdoors, where sidewalks, roads, and other terrain types must be distinguished. Consequently, outdoor navigation approaches commonly integrate semantic segmentation modules to identify walkable pedestrian regions [3]. However, many of these systems depend heavily on GPS localization [3] [11], limiting deployment in indoor environments. In contrast, our segmentation framework enables indoor and outdoor navigation without GPS.

Generalization to unknown environments also remains challenging. Many systems rely on prior map information from services such as Google Maps [11] or require building custom 3D maps of the environment beforehand [4], while others learn to follow previously teleoperated routes [6]. To improve navigation and obstacle avoidance in such environments, several approaches construct bird’s-eye-view (BEV) representations of the surroundings [3, 4], enabling the robot to maintain a spatial memory of nearby obstacles and free space. Multi-modal perception systems combining LiDAR and RGB sensing [4, 3] generally provide more robust obstacle detection and mapping than camera-only approaches [11, 6], particularly by reducing close-range blind spots and improving depth estimation. Unlike these systems, our framework does not require prior knowledge of the environment and incrementally builds a BEV map of the environment using onboard LiDAR and RGB sensors.

For obstacle avoidance and navigation planning, classical approaches typically rely on ground-plane costmaps to compute collision-free trajectories [5], while more recent methods train reinforcement learning policies in simulated BEV environments [3]. These recent learning-based approaches demonstrate promising generalization to unknown environments, but training often remains computationally expensive, requiring approximately 30 hours [3]. Our approach instead trains a navigation policy in approximately 10 minutes.

Handler Modeling. Guide dog mobility relies on continuous commands from the handler [8]. To reproduce this interaction in assistive robotics, prior work has explored rigid handle interfaces [12] [5] [4], and joystick-based controls [5] [4], allowing the handler to communicate directional intent and walking preferences. However, most existing guide robot systems oversimplify the handler’s physical presence during navigation by assuming a fixed geometric offset relative to the robot body [5]. Consequently, obstacle avoidance policies primarily plan trajectories for the robot itself, potentially generating paths that are safe for the robot but unsafe for the handler. Our work addresses this limitation by integrating joystick-based handler commands through a rigid guidance handle while explicitly modeling the handler’s dynamic spatial footprint within the obstacle avoidance and navigation loop.

3 Method

Rather than learning navigation end-to-end as a black-box policy, our approach constructs a human-interpretable bird’s-eye-view (BEV) representation of the environment, which serves as the policy input. Compared with a dense 3D map, the BEV representation is more memory-efficient while remaining suitable for downstream accessibility features, such as audio cues and natural-language scene descriptions (e.g., “obstacle ahead in 2 meters at 10 o’clock”). The navigation policy is trained offline in a GPU-accelerated 2D BEV simulator to follow randomly generated paths while avoiding obstacles and pedestrians. At run time, the onboard perception and control pipeline, illustrated in

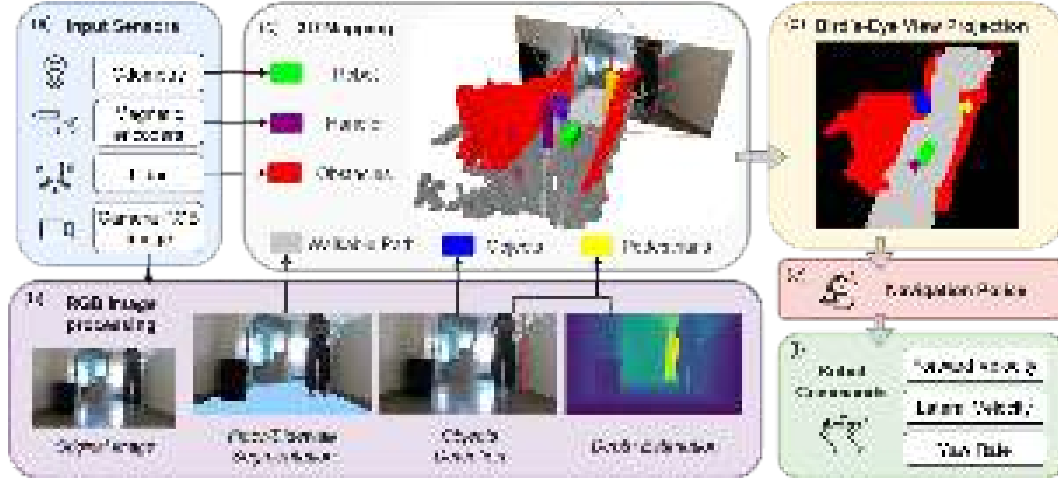


Figure 2: **Method Overview.** (a) RGB images, LiDAR scans, odometry, and handle encoder measurements are acquired. (b) RGB images are processed to segment the walkable path and detect objects and pedestrians, while depth estimation updates pedestrian positions. (c) The segmented path, detected objects and pedestrians, the robot and handler states, and LiDAR data are fused into a voxel-based 3D map. (d) The voxel map is converted into a bird’s-eye-view (BEV) representation. (e) The BEV map is provided to the trained navigation policy, which predicts the robot’s linear velocity and yaw rate commands. (f) The predicted commands are filtered by a local obstacle avoidance safety module before being sent to the robot.

Fig.2, acquires RGB images, LiDAR scans, odometry, and handle encoder measurements. RGB images are processed with NanoSAM [13, 14] to segment the walkable path and with YOLO [15] to detect objects and pedestrians. Pedestrian positions are estimated using Metric Depth Anything V2 [16], providing faster updates than relying solely on the LiDAR map. A 3D voxel-based map is constructed from temporally accumulated LiDAR scans, into which the segmented path, object detections, pedestrian positions, and the robot and handler states are fused. The map is then converted into a simplified BEV representation and serves as input to the trained navigation policy. Its predicted linear velocity and yaw rate commands are filtered by a local obstacle avoidance safety module before being scaled according to the handler’s preferred walking speed and sent to the robot.

3.1 Hardware

While the proposed framework does not depend on a specific robot platform, it was designed with low-cost quadruped robots in mind, such as the Unitree Go2 Air/Pro robot dog¹. We chose the Go2 for its affordability, front-facing camera and LiDAR, and wide availability. The robot does not come equipped with compute hardware that can run machine learning inference, which required us to add a compute module. The Nvidia Jetson Orin Nano² struck a good balance between price, size, and capabilities. In our experiments, it ran at 5 Hz our complete navigation pipeline, including voxel mapping, the segmentation and object detection net-

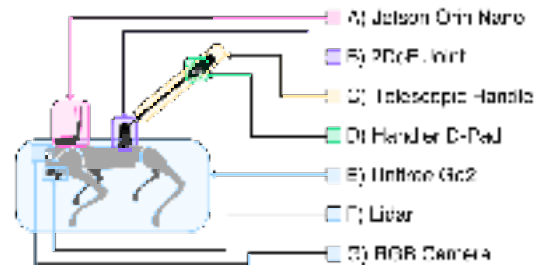


Figure 3: **Hardware Overview.** (A) The Nvidia Jetson Orin Nano runs navigation and perception systems; (B) the 2 degree-of-freedom joint is equipped with magnetic encoders that sense the handler’s position with respect to the dog; (C) the telescopic handle is adjustable to cater to different handler heights and preferences; (D) with the 5-way directional pad, the handler can start/stop the robot, indicate faster/slower walk speed, and desire to turn at the next opportunity; (E) the whole system is based on a Unitree Go2 Air/Pro robot dog, utilizing its (F) LiDAR and (G) front-facing RGB camera.

¹<https://www.unitree.com/go2>

²<https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin-nano-super-developer-kit/>

works, and the CNN-based policy. Guide dogs are typically held in the handler’s left hand, allowing the right hand to operate a white cane [8]. However, some left-handed handlers might prefer to switch sides. Flexible handler positioning enables the robotic guide dog to adapt its relative configuration when traversing narrow spaces, for example by temporarily moving ahead of the handler to avoid nearby obstacles. Supporting this flexibility required a custom physical interface; instead of a rigid harness, we use a telescopic handle that accommodates different handler-robot distance preferences and a custom 3D-printed 2 degrees-of-freedom (DoF) joint to securely attach the handle to the robot. The joint is equipped with magnetic encoder sensors to measure the azimuth and elevation angles of the handle in real-time. A five-way directional pad (D-pad) mounted at the end of the handle provides user commands, including pause/resume (center button), desired velocity adjustments (forward/backward), and turn requests (left/right). See Fig.3 for a diagram of the complete hardware configuration.

3.2 Navigation

In order to safely learn how to follow sidewalks and navigate indoor spaces, we built a sidewalk simulator similar to Sorokin et al. [3]³. The simulator was implemented in Taichi [9] to support GPU acceleration across devices, comparable to massively-parallel locomotion learning framework from Rudin et al. [17]. During training, the simulator generates random environments consisting of paths, intersections, walls, obstacles, and pedestrians surrounding the robot–handler pair. A target linear velocity is also randomly sampled for the robot to track. For more implementation details of the simulator, please see appendix section “Simulator Implementation”. In order to train a navigation policy in simulation, we express the problem as a Markov decision process with the following properties.

Observation Space. The observations are bird’s-eye view RGB images with a history of 2 additional previous observations. Please see Fig.2 for an annotated example. The desired velocity is overlaid into the image as vertical line, scaling its size with the handler’s desired velocity.

Action Space. The actions are forward and lateral velocity as well as yaw command.

Rewards. The reward function consists of seven terms: six binary collision penalties defined by the $[\text{dog}, \text{handler}] \times [\text{nonroad}, \text{wall}, \text{pedestrian}]$ collision matrix, where each term is activated when the corresponding collision occurs, and a linear velocity-tracking term $\mathbf{v}_t$. Let $\mathbf{v}_t$ be the handler’s desired forward velocity, $\mathbf{v}$ the current forward velocity, $\mathbf{V}_{\max}$ the maximum velocity, then

$$r_{\text{vel}} = \max(0, 1 - \frac{|\mathbf{v}_t - \mathbf{v}|}{V_{\max}}) \quad (1)$$

The reward term coefficients can be found in the appendix section “Reinforcement Learning”.

Policy Learning. Policy learning is implemented through Proximal Policy Optimization (PPO) from Rudin et al. [17].

3.3 Perception

The perception pipeline is designed to provide a real-time spatial representation of the robot, the handler, and their surrounding environment. To achieve this, the system combines data from on-board LiDAR, RGB camera, and magnetic encoders to construct a unified bird’s-eye view (BEV) representation of the scene. Images captured by the onboard camera are first rectified using intrinsic calibration parameters to remove lens distortion and fisheye effects.

BEV Representation. Inspired by BEVFusion [18], we construct a top-down semantic representation of the environment centered on the robot. The BEV map encodes walkable regions, obstacles,

³Sadly, their project page is no longer available, so we were not able to use their code if it was released.

185 detected objects and pedestrians, as well as the robot and handler positions. Object detections provide additional semantic context beyond obstacle occupancy, while explicitly representing pedestrians enables smoother interactions with moving entities. Fig.2 illustrates the different components of the generated BEV representation. The resulting BEV images are used as observations for the navigation policy described in Section 3.2.

190 **LiDAR Processing.** We use the LiDAR map produced by the robot’s onboard localization and mapping system and stream it in real time through the robot WebRTC interface. The resulting point cloud is motion-corrected to account for robot movement during LiDAR acquisition. The map is represented as a voxel-based 3D structure and downsampled into a lower-resolution occupancy grid before projection onto a 2D BEV plane. Voxels below a height threshold are removed to filter ground points while remaining robust to uneven terrain. Similarly, voxels above a maximum height threshold are discarded to ignore elevated structures such as ceilings or tree canopies that the robot and handler can safely pass under. The resulting occupancy grid represents surrounding obstacles and non-traversable regions and is continuously updated online as the robot moves through the environment.

200 **Walkable Path Segmentation.** Walkable path segmentation (e.g., indoor floors or outdoor sidewalks) is computed directly from RGB images. Our approach uses NVIDIA NanoSAM⁴, a distillation model optimized for Jetson devices based on MobileSAM [13]. NanoSAM generates segmentation masks from prompt points placed near the lower center of the image, assuming the robot is initially positioned on and facing a walkable path. The resulting segmentation mask is projected into the BEV representation.

206 **Object and Pedestrian Detection.** Objects and pedestrians are detected using Ultralytics YOLO26n [15] on rectified camera frames. Detections are associated with LiDAR points through geometric projection between the camera and LiDAR coordinate frames, then projected onto the 2D BEV map. Pedestrians are represented separately from generic obstacles to encourage smoother interactions with moving entities. To improve long-range pedestrian detection beyond the LiDAR occupancy map, we use monocular depth estimation with Depth Anything [16]. Depth predictions are calibrated with LiDAR measurements and are used to estimate pedestrian distance (see Appendix, “Pedestrian Depth Estimation”).

214 **Magnetic Encoder Processing.** The handler’s position relative to the robot is estimated using two magnetic encoders mounted at the base of the guiding handle. The measured azimuth, elevation, and handle length are used to compute the relative 3D position ($\mathbf{x}, \mathbf{y}, \mathbf{z}$) in the robot frame. This position is represented as a circular region in the BEV map, allowing the navigation policy to account for the handler during navigation.

219 4 Experiments

220 4.1 Experimental Setup

221 **Costmap Baseline.** As a baseline for evaluating our learned navigation policy, we implemented a classic costmap-based controller similar to Hwang et al. [5]. We first project the previously segmented walkable-path mask onto the ground plane using a calibrated homography transformation. From this ground projection, we generate five candidate trajectories corresponding to *extreme left*, *left*, *straight*, *right*, and *extreme right* motions. Each trajectory is modeled as a smooth Bézier curve to ensure forward and kinematically feasible motion. To evaluate a trajectory, we sample points along the curve and verify whether



Figure 4: **Example of Costmap-Based Trajectory Selection.** The segmented camera frame is projected onto the ground plane to generate trajectories. The center-right trajectory is selected as it has the lowest cost; ties are resolved in favor of the rightmost trajectory.

⁴<https://www.jetson-ai-lab.com/archive/vit/tutorial/nanosam.html>

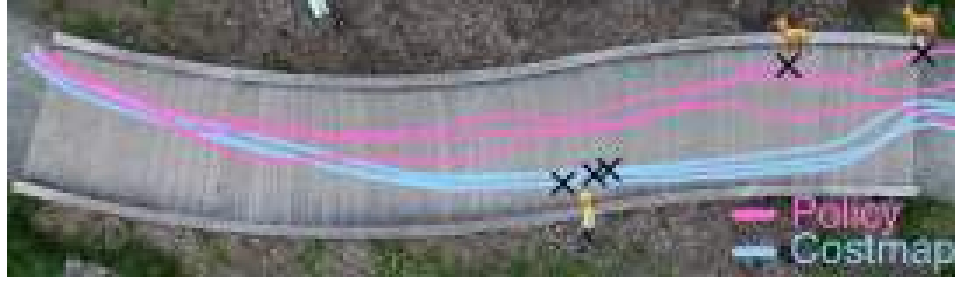


Figure 5: **Experiment - Path Following.** We compared the costmap controller (cyan) at fixed speed to the policy (pink) at variable speed. Starting at the left, both approaches had to cross the bridge and were timed at the start and finish. The costmap led the handler out of bounds for several meters (start marked on the map with X). The policy incurred 2 robot collisions (marked with X) at higher speeds but otherwise stayed center on the path.

they remain within the traversable region see Fig.4.

A traversal cost is then assigned based on how far along the trajectory it remains traversable. Trajectories that leave the walkable region early receive a high cost, whereas fully traversable trajectories receive a cost of zero. The trajectory with the minimum cost is selected for execution. In the case of equal costs, we prioritize straight trajectories over lateral alternatives and right-side trajectories over left-side ones. Finally, an intermediate goal point is extracted from the selected trajectory and transformed into world coordinates to provide the target position for the robot controller.

Control Loop For Policy Rollout. Once the policy is trained in simulation, the actor network is JIT-compiled for better performance. On the robot, 5 threaded loops are running at different frequencies, some governed by the publishing speed of that sensor, some limited by the hardware: (1) The LiDAR is accumulated into a voxel grid of 10cm resolution at 5Hz. This thread updates the walls in the BEV frame and provides the 3D geometry to which the floor and object segmentations are linked. (2) The odometry thread reads the robot’s position and orientation as well as the magnetic encoders to update the robot and handler positions in the BEV frame at 18Hz. (3) The camera thread receives images of 1280×720 px which are rectified to approximately 900×400 px. SAM and Yolo process the rectified frame and update object- and floor voxels for the BEV frame. Due to hardware constraints, this thread runs at 4Hz. (4) The policy extracts the latest BEV frame as a $200 \times 200 \times 3$ image, stacks it with the two previous observations to form a $200 \times 200 \times 9$ input, performs inference, and outputs a control command at 4 Hz (limited by the camera thread). (5) The control thread runs at 10Hz, repeating the last command from the policy while waiting for the next command.

4.2 Path Following

To verify that our system can follow paths without walls, solely relying on floor segmentation, we picked an outdoor path on an S-shaped wooden bridge in a park area (Fig.5, same route as shown in Fig.1). The starting position and orientation were picked so that the robot would need to turn at least twice to traverse safely with the handler following on the right side. We recorded 3 runs with the costmap controller with consistent speed and 3 runs of the policy with 3 different walking speeds to test how fast our system can traverse the course.

The costmap controller took an average of $21.3s \pm 1s$ and the policy ranged from 13.7s at the highest speed to 22.3s at the lowest. Since the costmap controller does not have any awareness of the handler’s relative position, every run incurred a handler collision with the boundary of the

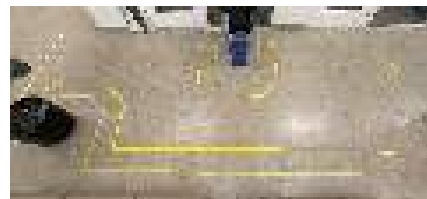


Figure 6: **Experiment - Obstacle Avoidance.** Milo and the handler started at the top right facing left and navigated around a rubbish bin and a planter. All costmap runs resulted in

course, and the robot followed a path unsafe for the handler for several meters. In the policy, we noticed 2 collisions of the robot at higher speeds for which we had to intervene and there were no handler collisions.

4.3 Static Obstacle Avoidance

We also tested turning around static obstacles, for which we set up a relatively tight turn for the robot & handler unit. We marked “comfort” areas on the floor around the obstacles with one line indicating 0.25m (dangerously close) and another line at 0.5m around obstacles. Following the same experimental protocol as before, we recorded 3 rollouts with the costmap controller and 3 runs with the policy. The completion time of the policy is slightly higher at $11.7s \pm 2s$ compared to the costmap $10.5s \pm 1s$. All costmap runs produced hard collisions of the handler with the first obstacle (rubbish bin) and 2/3 runs had the robot come dangerously close to the bin (under 0.25m). The policy runs did not include any dangerous handler trajectories, but in all rollouts, the robot walked dangerously close to the second obstacle and touched it in 2/3 runs. As before, we observed the policy being better at shielding the handler while incurring some collisions between the robot and the environment.

4.4 Additional Experiments

We also evaluated pedestrian avoidance with subsequent path recovery, as well as compliance with handler turn signals at intersections. These experiments are presented in the appendix sections ‘Experiment – Pedestrian’ and ‘Experiment – Turn Signal’.

5 Conclusion

In this work, we presented Milo, a fully self-contained, low-cost robotic guide dog designed to assist Blind and Low-Vision (BLV) users during navigation. The proposed system operates in previously unseen indoor and outdoor environments without requiring prior maps, external infrastructure, or internet connectivity. By combining lightweight onboard perception with a reinforcement-learning-based navigation policy, Milo learns to follow walkable paths while avoiding static and dynamic obstacles, including pedestrians. Our experiments demonstrate that Milo can reliably perform collaborative navigation in real-world environments. Compared to a classical costmap-based baseline, the learned policy reduced handler collisions and provided safer guidance for the user. Nevertheless, several limitations remain, as discussed below, motivating future work. By releasing the complete hardware and software stack as open source, we hope to support future research on robotic guide dogs and contribute to the development of more accessible mobility assistance technologies for the BLV community.

6 Limitations and Future Work

Our system has several limitations that motivate future work. First, the NanoSAM-based walkable-path segmentation assumes that the robot is already positioned on and facing a traversable surface, since the prompt points are placed near the lower center of the image. This limitation could be addressed by first using NanoOwl⁵ to localize semantic concepts such as sidewalks or floors and provide a bounding-box prompt for NanoSAM. Second, the learned policy does not always balance handler safety, robot safety, and social navigation. While it generally provides safer trajectories for the handler than the costmap baseline in simple environments, it may reduce the robot’s own clearance from obstacles. In addition, the robot tends to behave overly conservatively around pedestrians in crowded environments, despite people often approaching it out of curiosity. Future work could

⁵<https://github.com/NVIDIA-AI-IOT/nanoowl>

312 investigate socially-aware navigation policies that better optimizes these competing objectives. A
313 further limitation is that the current ground-projection method assumes flat terrain, and handling un-
314 even surfaces such as slopes and ramps remains future work. LiDAR map updates can also become
315 a bottleneck at higher walking speeds. Accessing raw LiDAR data through the DDS interface could
316 improve update rates, at the cost of increased processing complexity. Finally, the current framework
317 is limited to sidewalk and floor navigation and cannot cross streets. Future work could extend the
318 system to more complex urban environments by incorporating pedestrian crossing, traffic light, and
319 vehicle detection, while also providing richer voice feedback to improve the handler’s understanding
320 of the surroundings, similar to the approach proposed by Hayamizu et al. [7].

References

- [1] R. R. Bourne, J. D. Steinmetz, S. Flaxman, et al. Trends in prevalence of blindness and distance and near vision impairment over 30 years: an analysis for the global burden of disease study. *The Lancet Global Health*, 9(2):e130–e143, 2021. ISSN 2214-109X. doi:10.1016/S2214-109X(20)30425-3. URL [https://www.thelancet.com/journals/langlo/article/PIIS2214-109X\(20\)30425-3/fulltext](https://www.thelancet.com/journals/langlo/article/PIIS2214-109X(20)30425-3/fulltext).
- [2] CNIB Foundation. Guide the way, 2025. URL <https://www.guidedog.org/gd/about-us/about-the-guide-dog-foundation.aspx>. Accessed: 2026-05-28.
- [3] M. Sorokin, J. Tan, C. K. Liu, and S. Ha. Learning to navigate sidewalks in outdoor environments. *IEEE Robotics and Automation Letters*, 7(2):3906–3913, 2022.
- [4] S. Cai, A. Ram, Z. Gou, M. A. W. Shaikh, Y.-A. Chen, Y. Wan, K. Hara, S. Zhao, and D. Hsu. Navigating real-world challenges: A quadruped robot guiding system for visually impaired people in diverse environments. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pages 1–18, 2024.
- [5] H. Hwang, T. Xia, I. Keita, K. Suzuki, J. Biswas, S. I. Lee, and D. Kim. System configuration and navigation of a guide dog robot: Toward animal guide dog-level guiding work. *arXiv preprint arXiv:2210.13368*, 2022.
- [6] H. Hwang, S. Yang, J. S. Monon, N. A. Giudice, S. I. Lee, J. Biswas, and D. Kim. GuideNav: User-informed development of a vision-only robotic navigation assistant for blind travelers, 2025. URL <http://arxiv.org/abs/2512.06147>.
- [7] Y. Hayamizu, D. DeFazio, H. Mehta, Z. Altaweel, J. Choe, C. Lin, J. Juettner, F. Xiao, J. Blackburn, and S. Zhang. From woofs to words: Towards intelligent robotic guide dogs with verbal communication, 2026. URL <http://arxiv.org/abs/2603.12574>.
- [8] H. Hwang, H.-T. Jung, N. A. Giudice, J. Biswas, S. I. Lee, and D. Kim. Towards robotic companions: Understanding handler-guide dog interactions for informed guide dog robot design, 2024. URL <http://arxiv.org/abs/2402.06790>.
- [9] Y. Hu, T.-M. Li, L. Anderson, J. Ragan-Kelley, and F. Durand. Taichi: a language for high-performance computation on spatially sparse data structures. *ACM Transactions on Graphics (TOG)*, 38(6):201, 2019.
- [10] J. Chen and B. Zhang. Exploration and navigation in unknown environments for guide dog robots. In *2025 9th International Conference on Robotics and Automation Sciences (ICRAS)*, pages 48–326, 2025. doi:10.1109/ICRAS65818.2025.11108808. URL <https://ieeexplore.ieee.org/document/11108808>. ISSN: 2694-3506.
- [11] J. Viteri and C.-H. G. Li. Autonomous sidewalk navigation featuring end-to-end RGB-d dual-ConvNet steering. In *2024 IEEE International Conference on Advanced Intelligent Mechatronics (AIM)*, pages 703–708. IEEE, 2024. ISBN 979-8-3503-5536-9. doi:10.1109/AIM55361.2024.10637141. URL <https://ieeexplore.ieee.org/document/10637141/>.
- [12] J. T. Kim, W. Yu, Y. Kothari, J. Tan, G. Turk, and S. Ha. Transforming a quadruped into a guide robot for the visually impaired: Formalizing wayfinding, interaction modeling, and safety mechanism. *arXiv preprint arXiv:2306.14055*, 2023.
- [13] C. Zhang, D. Han, Y. Qiao, J. U. Kim, S.-H. Bae, S. Lee, and C. S. Hong. Faster segment anything: Towards lightweight sam for mobile applications. *arXiv preprint arXiv:2306.14289*, 2023.
- [14] NVIDIA. Nanosam. https://www.jetson-ai-lab.com/archive/vit/tutorial_nanosam.html, 2024. Accessed: 2026-05-28.

- 366 [15] G. Jocher and J. Qiu. Ultralytics yolo26, 2026. URL [https://github.com/ultralytics/](https://github.com/ultralytics/ultralytics)
367 [ultralytics](https://github.com/ultralytics/ultralytics).
- 368 [16] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao. Depth anything v2.
369 *arXiv:2406.09414*, 2024.
- 370 [17] N. Rudin, D. Hoeller, P. Reist, and M. Hutter. Learning to walk in minutes using massively
371 parallel deep reinforcement learning. arxiv. *arXiv preprint arXiv:2109.11978*, 2021.
- 372 [18] T. Liang, H. Xie, K. Yu, Z. Xia, Z. Lin, Y. Wang, T. Tang, B. Wang, and Z. Tang. Bevfusion: A
373 simple and robust lidar-camera fusion framework. *Advances in Neural Information Processing*
374 *Systems*, 35:10421–10434, 2022.
- 375 [19] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo. Segformer: Simple and
376 efficient design for semantic segmentation with transformers. *CoRR*, abs/2105.15203, 2021.
377 URL <https://arxiv.org/abs/2105.15203>.



Figure 7: **Comparison of NanoSAM and SegFormer walkable-surface segmentation.** Columns show the input RGB image, NanoSAM prediction, and SegFormer prediction, respectively. Rows correspond to an indoor floor, an outdoor sidewalk, a tiled pedestrian area, a park path, a sidewalk partially covered by snow, and a camera frame where the robot is facing a wall. Both methods perform well on standard indoor-floor and outdoor-sidewalk examples. NanoSAM produces more consistent segmentations on tiled surfaces and better generalizes to park paths and snow-covered sidewalks. However, when the robot is facing a wall, NanoSAM incorrectly segments the wall as walkable because of its prompt-based formulation, whereas SegFormer successfully identifies the small visible floor region. These examples highlight the trade-off between the stronger generalization capabilities of NanoSAM and the orientation robustness of SegFormer.

As an alternative to NanoSAM, we evaluated walkable-surface segmentation approaches based on SegFormer [19], including models fine-tuned for outdoor sidewalk segmentation⁶ and indoor floor segmentation⁷. These models directly predict dense segmentation masks from RGB images, en-

⁶<https://huggingface.co/tobiasec/segformer-b0-finetuned-segments-sidewalk>

⁷<https://huggingface.co/nvidia/segformer-b0-finetuned-ade-512-512>

382 abling the identification of walkable areas without requiring user-defined prompt points. Unlike
 383 NanoSAM, SegFormer does not rely on assumptions about the robot’s initial orientation relative to
 384 the path, making it more robust in situations where the robot is not facing a traversable surface, as
 385 illustrated by the last row of Fig. 7.

386 However, separate models are required for indoor-floor and outdoor-sidewalk segmentation, neces-
 387 sitating model switching when transitioning between indoor and outdoor environments. In addi-
 388 tion, we observed degraded performance in scenarios that differ from the training distribution, such
 389 as snow-covered sidewalks, tiled surfaces, and park paths. Fig. 7 highlights the complementary
 390 strengths and limitations of the two approaches. We opted for SAM/NanoSAM because the main
 391 downside is easy to address: starting the robot not directly facing a wall.

392 Appendix B Pedestrian Depth Estimation

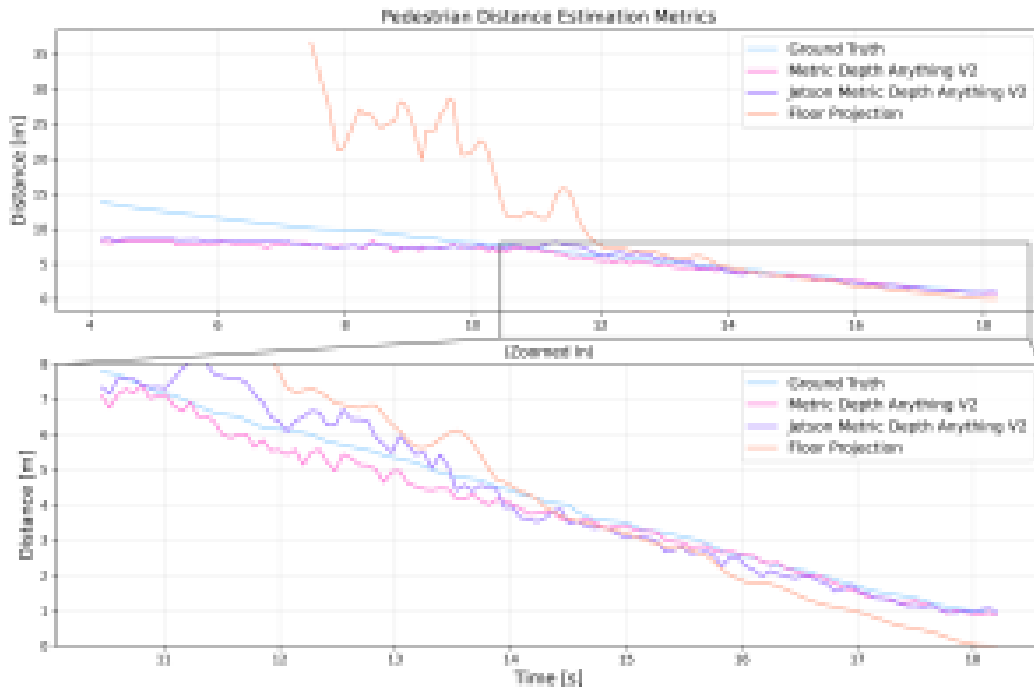


Figure 8: **Estimated distance to a pedestrian as a function of time for several camera-based localization approaches.** All methods were evaluated on the same video recording of a person walking toward the robot. Ground-truth distances were obtained using an AirTag carried by the pedestrian and were interpolated to produce a smooth reference trajectory. The figure compares three methods for estimating the distance of pedestrians detected by YOLO: a floor-projection approach that projects the center of the lower edge of each YOLO bounding box onto the ground plane using the camera calibration, a fine-tuned version of Depth Anything V2 for indoor metric depth estimation calibrated using LiDAR measurements, and a Jetson-optimized implementation based on the IRCVLab TensorRT pipeline using the same fine-tuned model weights and LiDAR calibration. The floor-projection method exhibits significant noise at longer ranges due to frame-to-frame variations in bounding box localization, whereas depth-based methods provide smoother and more accurate estimates. The two Metric Depth Anything V2 implementations achieve comparable accuracy across the evaluated range, while the Jetson-optimized version provides substantially higher inference speed, enabling real-time deployment on Milo.

393 The LiDAR-based obstacle map used by the navigation system is updated at a relatively low fre-
 394 quency and covers only a local region around the robot. While sufficient for static obstacle avoid-

ance, this representation can introduce latency when tracking moving pedestrians. To improve responsiveness, we evaluated two camera-based pedestrian localization methods.

The first method uses YOLO pedestrian detections and projects the center of the lower edge of each bounding box onto the ground plane using a precomputed camera calibration. This approach is lightweight and suitable for onboard deployment, but the resulting position estimates are often noisy due to frame-to-frame variations in the bounding box location.

The second method combines YOLO detections with monocular depth estimation using a fine-tuned version of Depth Anything V2 for indoor metric depth estimation⁸ [16]. Although the model directly predicts metric depth, its estimates are further calibrated using LiDAR measurements to improve consistency with the robot’s sensing setup. The pedestrian distance is estimated from the closest 10% of depth values within each detected bounding box. Although the model is fine-tuned for indoor scenes, preliminary testing showed that it also produced reliable depth estimates in our outdoor environment over the distance range considered in this work. Compared with the floor-projection approach, this method produces smoother and more accurate distance estimates, particularly at longer ranges.

To enable real-time deployment on the Jetson Orin, we adopted the TensorRT optimization pipeline proposed by IRCVLab⁹ while replacing the original model weights with those of the fine-tuned Depth Anything V2 model for indoor metric depth estimation. This optimized implementation provides distance estimates that are comparable in accuracy to the fine-tuned metric Depth Anything V2 implementation while increasing the inference rate from 2.64 FPS to 24.02 FPS, making it suitable for real-time onboard deployment on Milo.

Fig. 8 compares the different methods for estimating the distance between a pedestrian and the robot. Ground-truth measurements were obtained using an AirTag carried by a person walking toward the robot, and all methods were evaluated on the same video recording. The floor-projection approach exhibits considerable noise, particularly at longer distances, due to frame-to-frame variations in the detected bounding boxes. Both implementations of the fine-tuned Depth Anything V2 model for indoor metric depth estimation produce smoother and more accurate distance estimates. Their performance is similar over the evaluated range, while the TensorRT-optimized Jetson implementation achieves a significantly higher inference rate (approximately 24.02 FPS compared with 2.64 FPS), making it suitable for real-time onboard deployment on Milo.

Appendix C Local Obstacle Safety Filter

TODO

Appendix D Simulator Implementation

Our birds-eye-view simulator, implemented in Taichi [9], constructs scenes like this: Roads are random lines and Bezier curves drawn in OpenCV (only on episode reset) that can be rotated and combined for tee and cross intersections. The size of the roads is sampled in the range of sidewalk widths allowed in the civil code of our city. 2D Perlin noise is applied to the road map to approximate incomplete floor segmentation. The walls are road contours that are randomly sampled (to allow for segments with 0, 1, or 2 walls) and drawn on top of the roads. The robot model is spawned at one of the points making up the road center lines, slightly offset and rotated towards the far end of the map. The camera is always tracking the center of the robot with the robot facing upward (i.e. the rest of the map is rotated, zoomed, and cropped to keep a 8m or 4m local area around the robot

⁸<https://huggingface.co/depth-anything/Depth-Anything-V2-Metric-Indoor-Small-hf>

⁹<https://github.com/IRCVLab/Depth-Anything-for-Jetson-Orin>

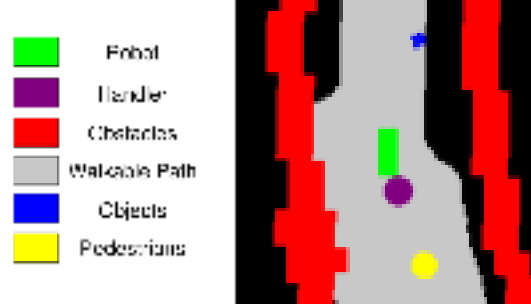


Figure 9: **Example Simulator Frame.** Visualization of the simulated environment. The traversable road is shown in grey (with intentional gaps to mimic imperfect floor segmentation), walls in red, handler in purple, pedestrian in yellow, and obstacle in blue.

in view¹⁰). On top of the scene, we draw the handler (a circle at a random position parameterized by azimuth, elevation, and length of the handle), obstacles (randomly rotated boxes, with positions sampled between the center line and the edge of the road), and pedestrians (circles, following the path segments at a fixed left/right offset and random fixed speed).

Appendix E Reinforcement Learning

In addition to the main reward terms described in the main body of the paper, we also include the following auxiliary reward terms at lower reward coefficients to stabilize training: A jerk penalty, expressed as the squared difference between the last action $\mathbf{a}_{t-1}$ and the current action $\mathbf{a}_t$.

$$\mathbf{r}_{\text{jerk}} = \|\mathbf{a}_{t-1} - \mathbf{a}_t\|^2 \quad (2)$$

We also included a gradual distance penalty $\mathbf{r}_{\text{circle}}$ to increase awareness of obstacles. The term is constructed by counting the number of pixels of walls, obstacles, and pedestrians that are within a circle around the robot and handler. The full reward consists of the following reward terms and their coefficients:

- $\mathbf{r}_{\text{coll_wall}} \square -4$ for collisions between dog/handler and wall,
- $\mathbf{r}_{\text{coll_ped}} \square -3$ for collisions between dog/handler and pedestrians,
- $\mathbf{r}_{\text{coll_obst}} \square -2$ for collisions between dog/handler and obstacles,
- $\mathbf{r}_{\text{coll_off}} \square -0.1$ for dog/handler leaving the visible road,
- $\mathbf{r}_{\text{vel_fwd}} \square 2$ for tracking the forward velocity,
- $\mathbf{r}_{\text{vel_aux}} \square 0.5$ for tracking the lateral velocity and yaw rate of zero,
- $\mathbf{r}_{\text{jerk}} \square -0.2$ for quantifying the difference between the previous and current action, and
- $\mathbf{r}_{\text{circle}} \square -0.2$ for measuring the amount of obstacles/walls/pedestrians in the vicinity.

The hyperparameters for PPO are as follows: see Tab.2

Appendix F Experiment – Pedestrian

Our intention was to evaluate the comfort (minimum distance) and number of handler and robot collisions in an obstacle course consisting of 2 collaborators walking towards the robot and handler in a corridor at a fixed pace, but in practice, we found little repeatability in this setup

¹⁰Ideally, we would have wanted the larger 8m viewport for smoother obstacle and pedestrian avoidance, but we found empirically that the L2 LiDAR on the Go2 is not reliable enough for this distance. In practice, we used the 4m view port for all experiments. We leave better integration of depth estimation networks to improve the LiDAR accuracy not just for pedestrians but also for the path geometry to future work.

462 between experiments. Qualitatively, Milo was able to avoid oncoming pedestrians but struggled
463 with pedestrians overtaking from behind. The latter scenarios sometimes led to side-stepping
464 behaviors when the overtaking pedestrian first came in sight and until they were at least a meter
465 past Milo. We believe this behavior can be addressed by modulating the lateral and yaw response
466 when pedestrians suddenly appear within a 1m radius. We included samples of unplanned outdoor
467 pedestrian avoidance in the supplementary video:

468

469 <https://youtu.be/hCJE40Roizo>

Table 2: PPO and CNN Hyperparameters

Parameter	Value
<i>PPO Configuration</i>	
Training iterations	2000
Steps per environment	64
Parallel environments	10
Clip parameter	0.2
Learning epochs	5
Mini-batches	4
Learning rate	3×10^{-4}
Gamma	0.99
Lambda (GAE)	0.95
Entropy coefficient	0.01
Desired KL	0.01
Max grad norm	1.0
<i>Network Configuration</i>	
Observation normalization	True
Output channels	[32, 64, 64]
Kernel sizes	[8, 4, 3]
Strides	[4, 2, 1]
Activation	ELU